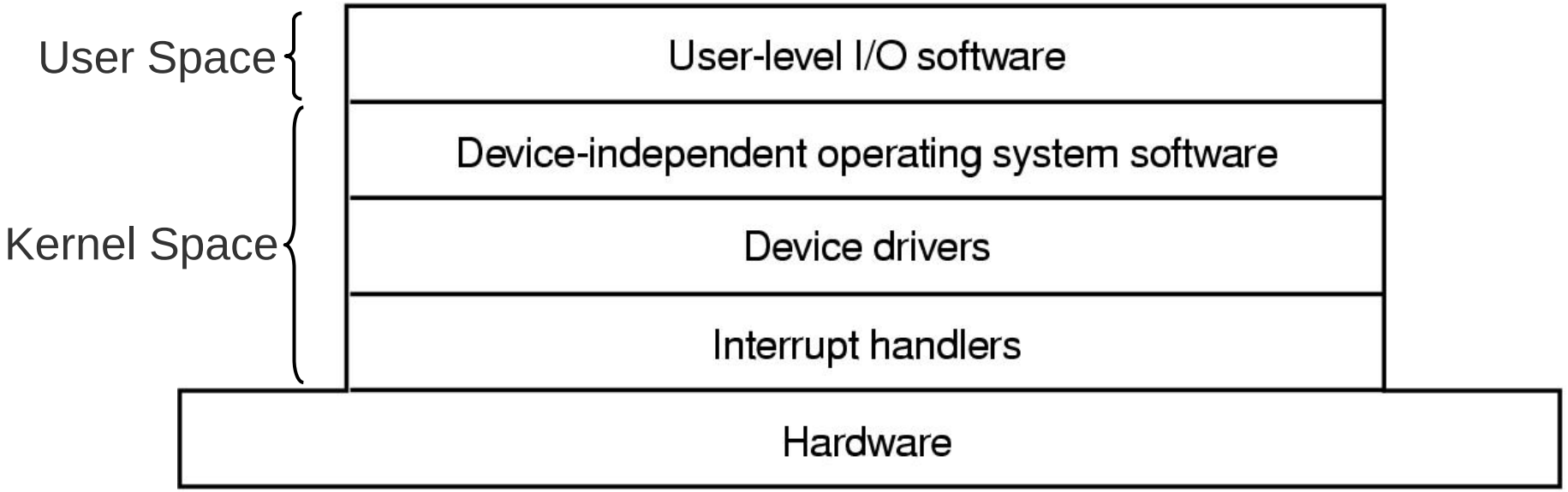
Chapter 5b
Input/Output

- Principles of I/O hardware
- Principles of I/O software
- I/O software layers
- Disks

I/O Software Layers



Layers of the I/O Software System

Interrupt Handlers (1)

- Interrupt handlers are best hidden
  - have driver starting an I/O operation block until interrupt notifies of completion
- Interrupt procedure does its task
  - then unblocks driver that started it

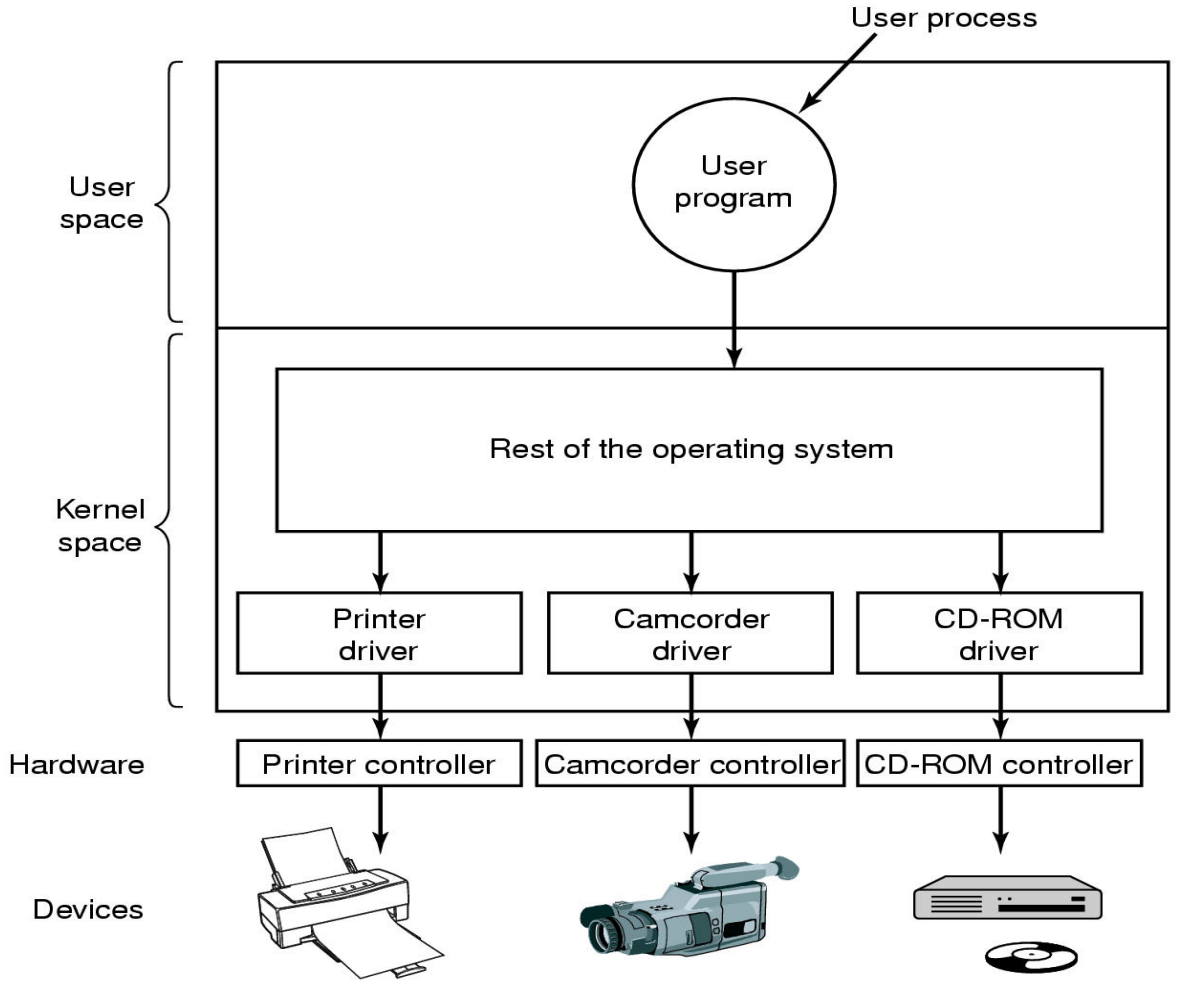
Interrupt Handlers (2)

- Steps must be performed in software after interrupt completed
  1. Save regs not already saved by interrupt hardware
  2. Set up context for interrupt service procedure
  3. Set up stack for interrupt service procedure
  4. Ack interrupt controller, reenable interrupts
  5. Copy registers from where saved
  6. Run service procedure
  7. Choose which process to run next.
  8. Set up MMU context for process to run next
  9. Load new process' registers
  10. Start running the new process

Device Drivers (1)

- Encapsulates routines that handle low-level device operations
- Generally written by device manufacturers
  - Not necessarily by OS writers
- A driver can be written for more than one OS
- Each driver can handle one type of device
- We need to have a well-defined model of what a driver does and how it integrates with the rest of the OS.
  - This is because, OS implementers and drivers implementers may be different.
- OSs usually classify drivers into two classes
  - Drivers for block devices
  - Drivers for character devices

Device Drivers (2)



- Logical position of device drivers is shown here
- Communications between drivers and device controllers goes over the bus

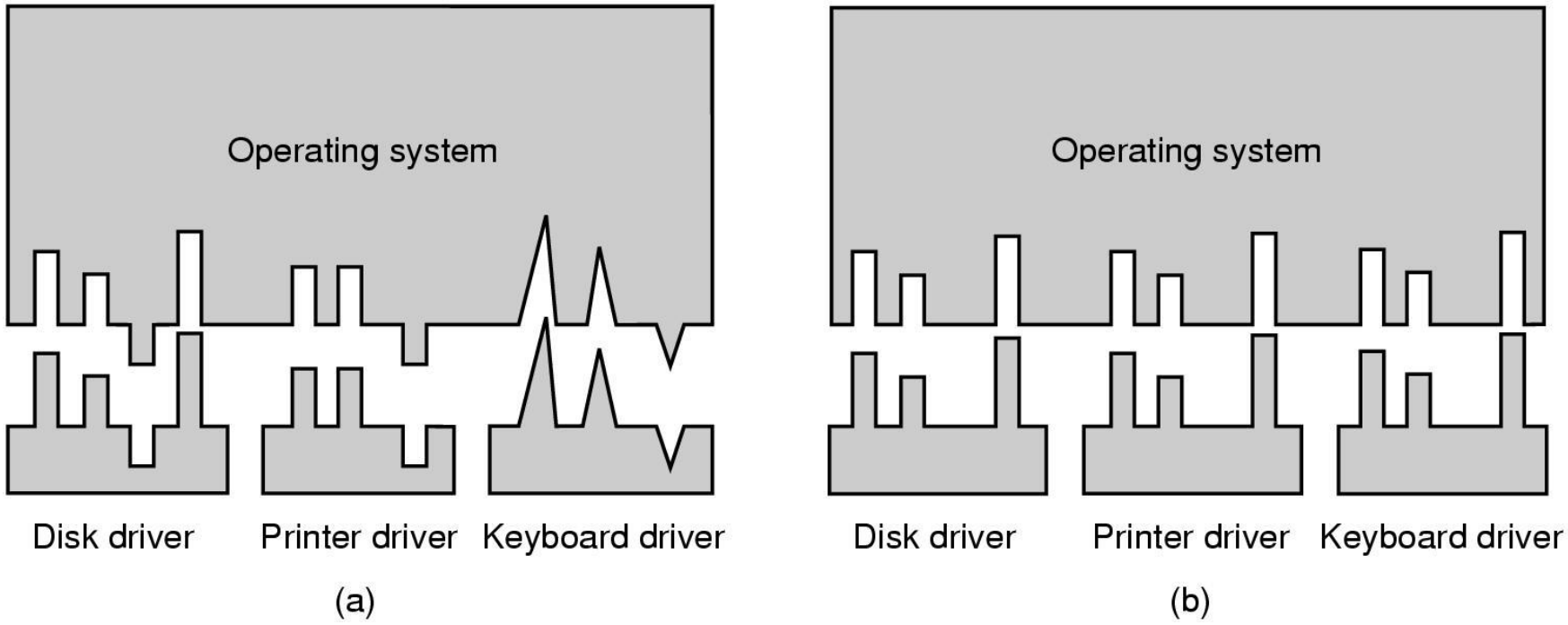
Device Drivers (3)

- Device Driver Functions
  - Accept abstract read and write requests from rest of the OS (device-independent part).
    - Translate from abstract terms to concrete terms.
      - Example: A disk driver translates from a linear block address to a physical head, track, sector, and cylinder number.
  - Initialize the devices if necessary
  - Check if the device is currently in use for some other request.
    - If so, enqueue the request.
  - Issue sequence of commands to control the device
  - Check errors

Device-Independent I/O Software

- Functions of the device-independent I/O software
  - Uniform interfacing for device drivers
  - Buffering
  - Error reporting
  - Allocating and releasing dedicate devices
  - Providing a device-independent block size

Uniform Interfacing (1)



(a) Without a standard driver interface

(b) With a standard driver interface

Uniform Interfacing (2)

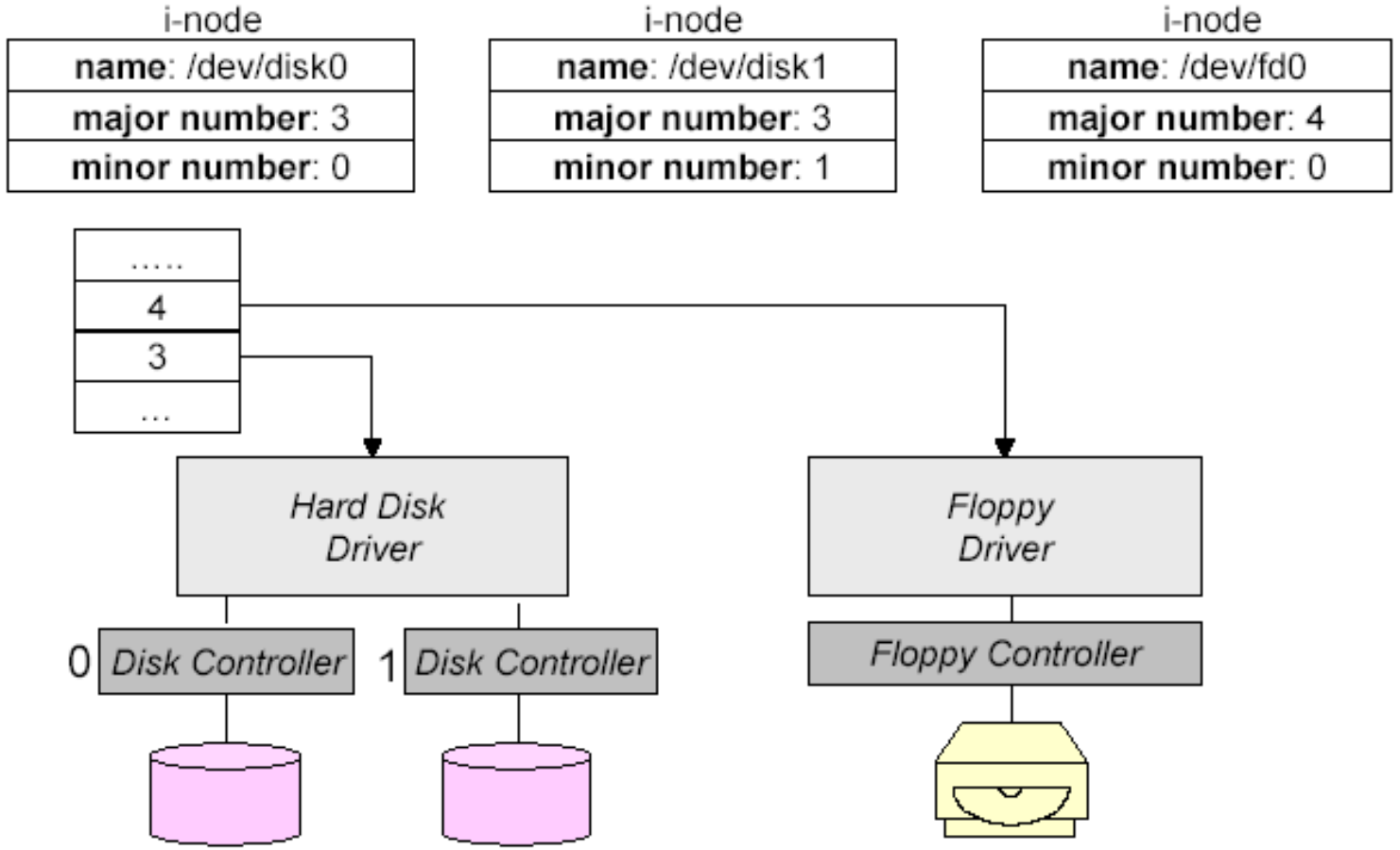
- Drivers need to have uniform interface. The benefits are:
  - The driver implementers know what is expected from them
  - The OS implementers can developed device-independent I/O function on top of a well-defined lower-layer driver interface.
    - They know which functions each driver implements and the pro-types of these function.

Uniform Interfacing (3)

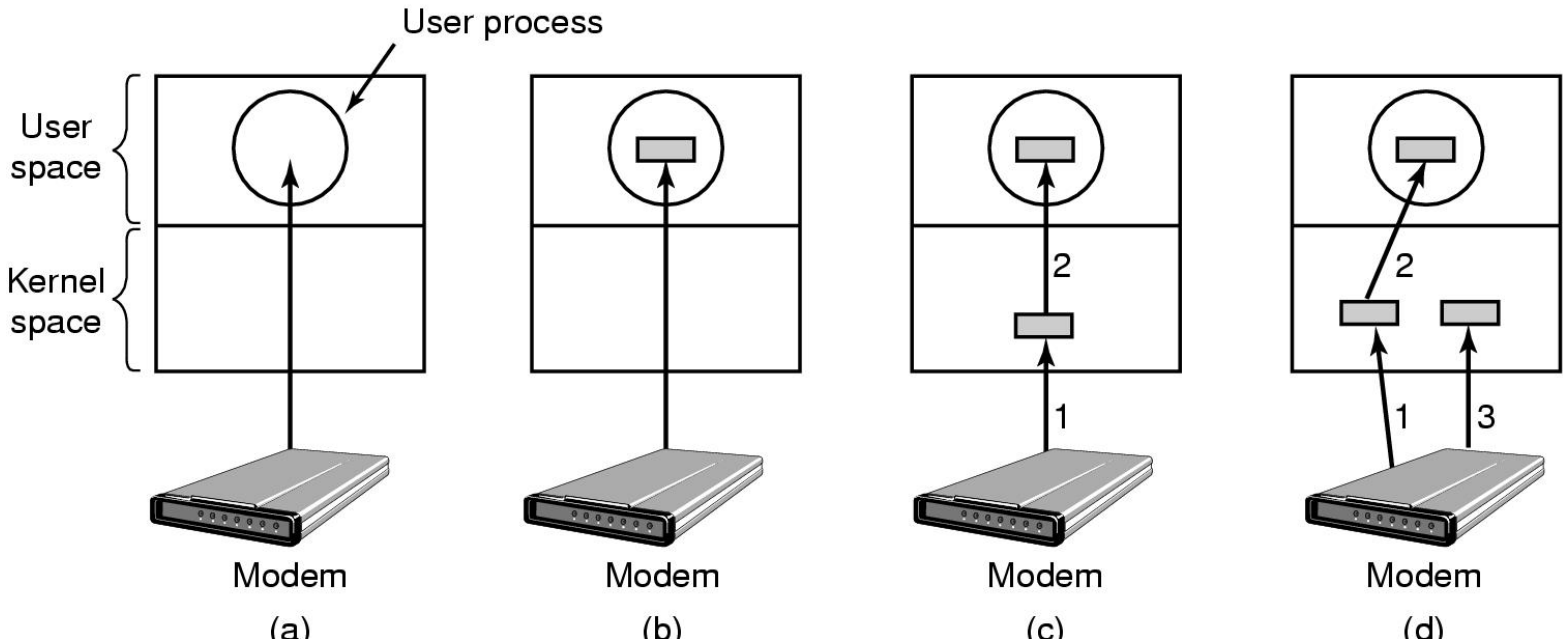
- In Unix, devices are modeled as special files.
  - They are accessed through the use of system calls such as open(), read(), write(), close(), ioctl(), etc.
  - A file name is associated with each device.
- Major device number locates the appropriate driver.
  - Minor device number (stored in i-node) is passed as a parameter to the driver in order to specify the unit to be read or written.
- The usual protection rules for files also apply to I/O devices

Uniform Interfacing (4)

- Uniform Interfacing: Mapping symbolic I/O device names to their appropriate drivers

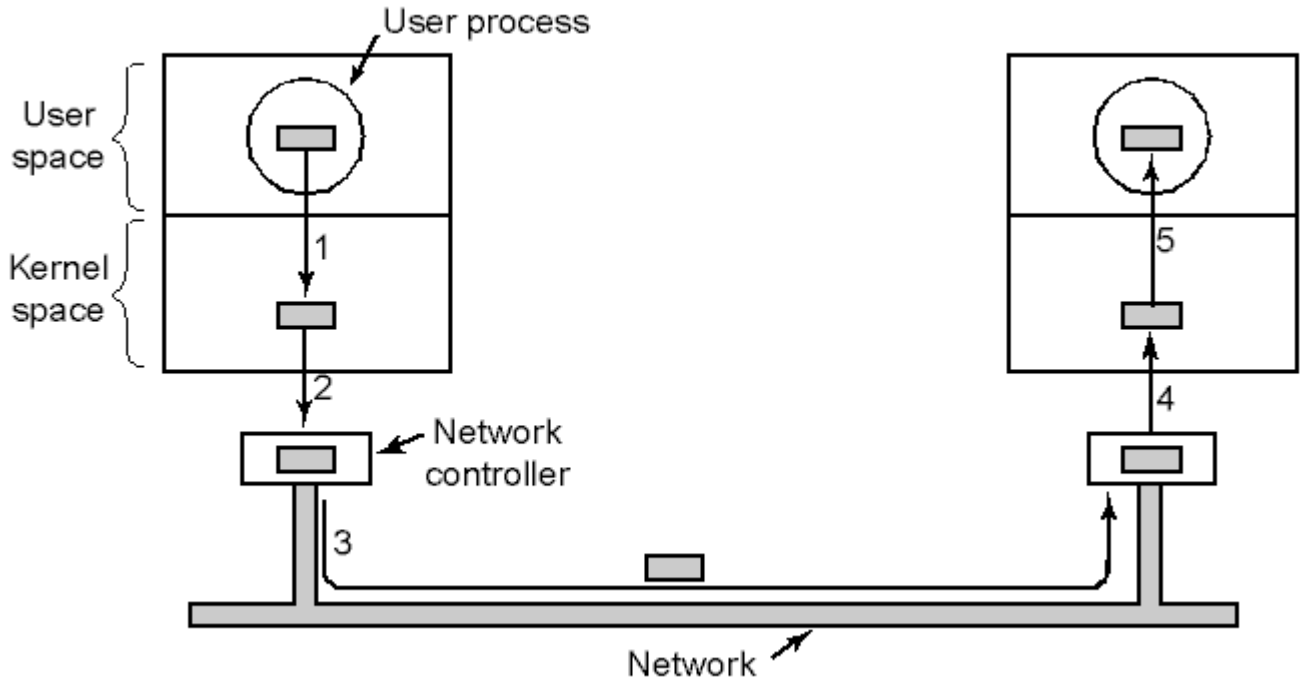


Buffering (1)



(a) Unbuffered input
(b) Buffering in user space
(c) Buffering in the kernel followed by copying to user space
(d) Double buffering in the kernel

Buffering (2)



Networking may involve many copies

Error Reporting

- Some errors are handled by device-controllers
  - Example: Checksum incorrect, re-correct the block by using redundant bits
- Some by device derivers
  - Example: disk block could not be read, re-issue the read operation for block from disk
- Some by the device-independent software layer of OS.
  - Programming errors
    - Example: Attempt to write to a read-only device
  - Actual I/O errors
    - Example: The camcorder is shut-off, therefore we could not read. Return an indication of this error to the calling application.

Allocating Dedicated Devices

- Do not allow concurrent accesses to dedicated devices such as CD-RWs.

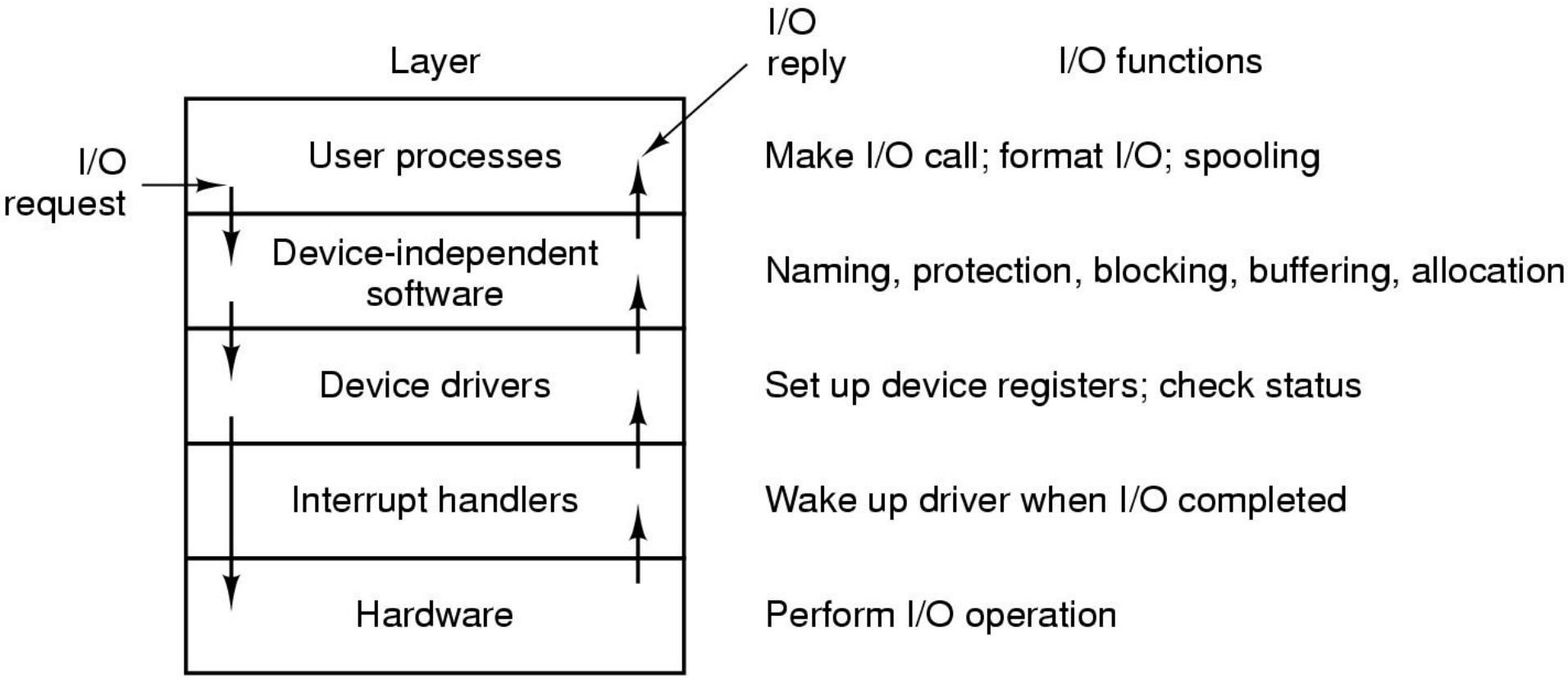
Device-independent Block Size

- There are different kind of disk drives. Each may have a different physical sector size.
- A file system uses a block size while mapping files into logical disk blocks.
- This layer can hide the physical sector size of different disks and can provide a fixed and uniform disk block size for higher layers, like the file system
  - Several sectors can be treated as a single logical block.

User-Space I/O Software

- This includes
  - The I/O libraries that provides the implementation of I/O functions which in turn call the respective I/O system calls.
    - These libraries also implemented formatted I/O functions such as printf() and scanf()
  - Some programs that does not directly write to the I/O device, but writes to a spooler.

Summary of I/O Software



Layers of the I/O system and the main functions of each layer

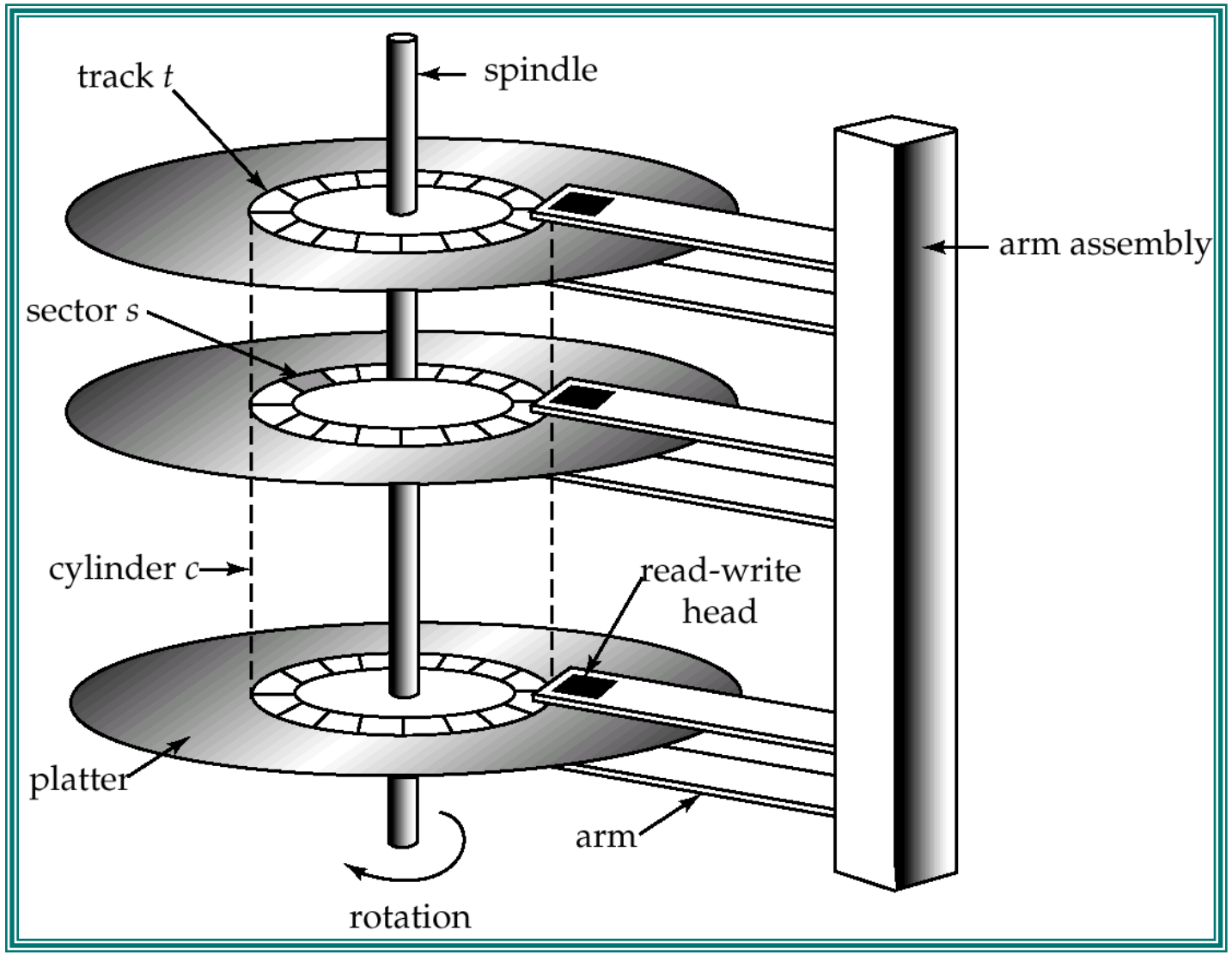
Disks

- Disks have many types
  - Magnetic disks
    - Hard disk
    - Floppy disk
  - Optical disks
    - CD-ROM
    - CD-Writable
    - CD-Recordable
    - DVDs

Magnetic Disks (1)

- Organized into cylinders
  - Each cylinders contains tracks
    - Each track is divided into a number of sectors
      - The sector size is usually 512 bytes.
- There are IDE, SATA, SCSI, SAS and Fiber Channel hard disks.

Magnetic Disks (2)

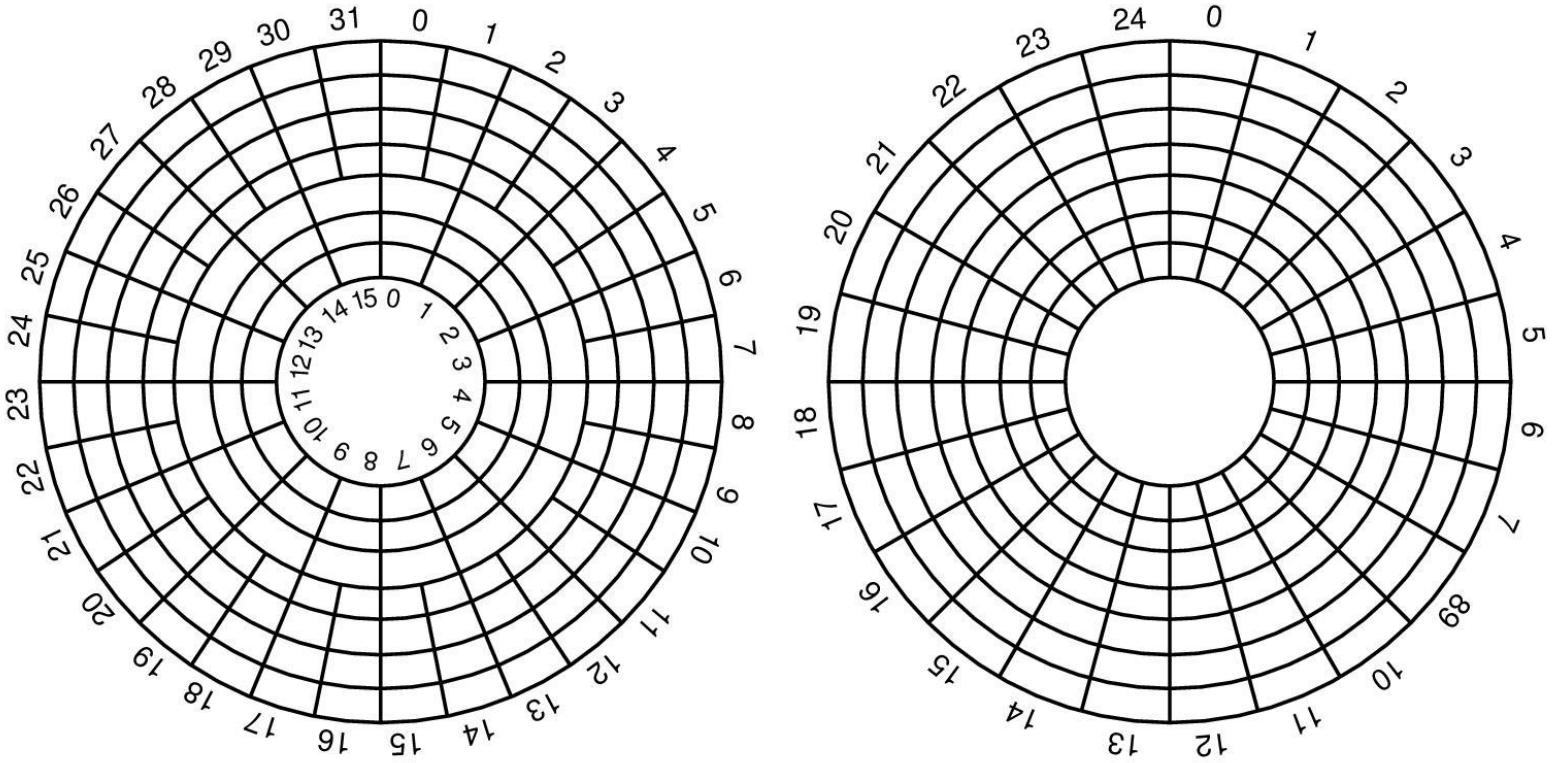


Magnetic Disks (3)

Parameter	IBM 360-KB floppy disk	WD 18300 hard disk
Number of cylinders	40	10601
Tracks per cylinder	2	12
Sectors per track	9	281 (avg)
Sectors per disk	720	35742000
Bytes per sector	512	512
Disk capacity	360 KB	18.3 GB
Seek time (adjacent cylinders)	6 msec	0.8 msec
Seek time (average case)	77 msec	6.9 msec
Rotation time	200 msec	8.33 msec
Motor stop/start time	250 msec	20 sec
Time to transfer 1 sector	22 msec	17 µsec

Disk parameters for the original IBM PC floppy disk and a Western Digital WD 18300 hard disk

Magnetic Disks (4)



- Physical geometry of a disk with two zones
- A possible virtual geometry for this disk

Disk Formatting (1)

- After manufacturing, there is no information on the disk
  - Just empty bits
- Each platter need to have
  - A low-level format
  - A high-level format

before disk can be used

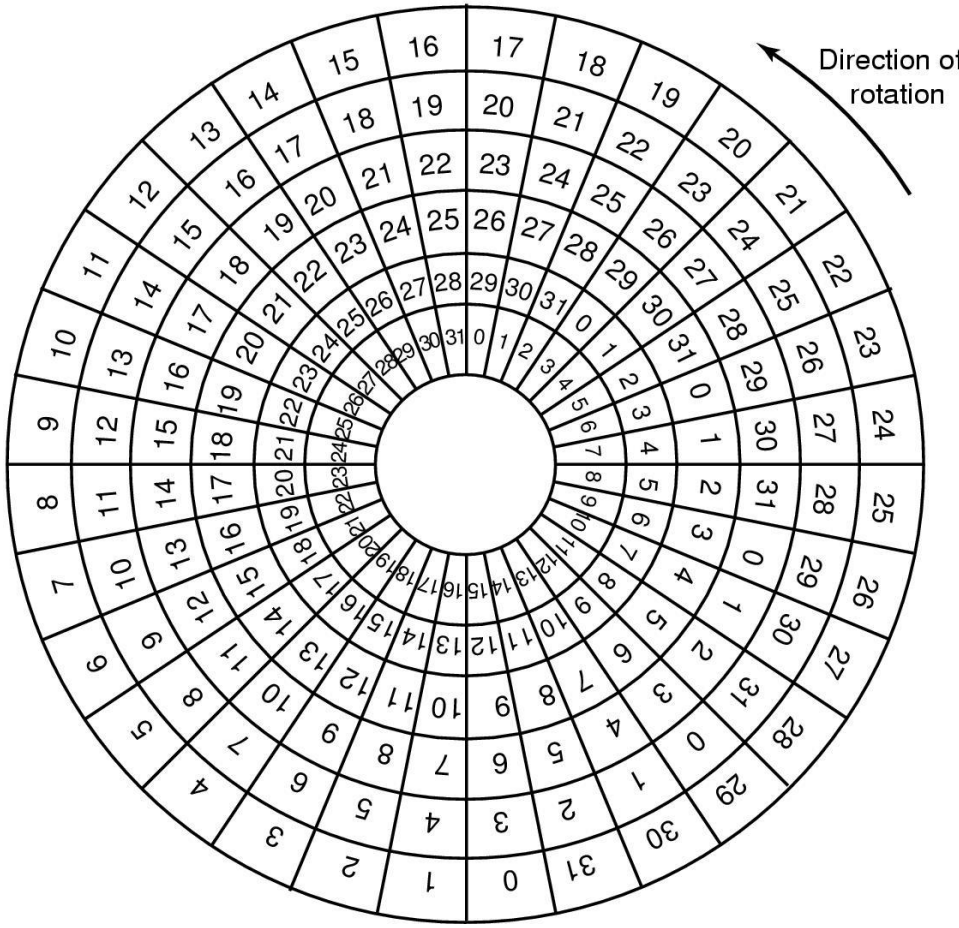
Disk Formatting (2)

- Each track should have formatted in the following way:
  - Sectors and inter-sector-gap between them
  - A sector will be have format like the following:
    - Preamble – data (512 bytes) – Checksum



-certain bit pattern
- cylinder number
-sector number

Cylinder Skew



Example

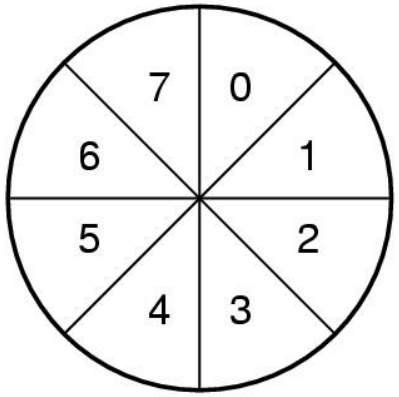
- 10,000rpm disk drive (6ms/round)
- 300 sectors per track (20 μs/sector)
- Seek time: 800 μs
- Cylinder skew: 40secotrs

An illustration of cylinder skew(柱面斜进)

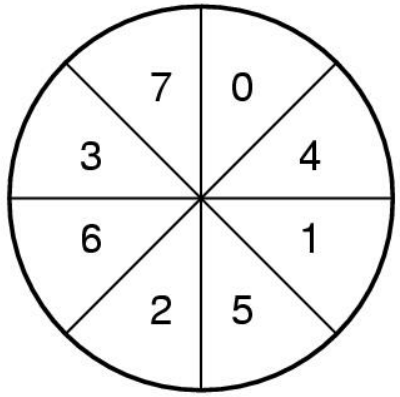
Interleaving (1)

- If we have ONE sector buffer in the disk controller:
  - After we have transferred one sector of data from hard-disk to controller buffer:
    - We will copy the controller buffer to the memory
    - During this time, the disk-head will pass the start of the next sector.
    - Therefore, the next logical sector, should not be the next physical sector in hard-disk.
      - There should be some interleaving.

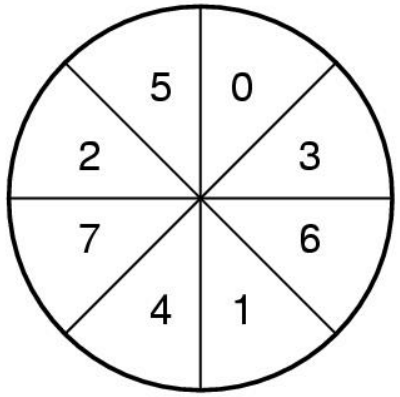
Interleaving (2)



(a)



(b)



(c)

- No interleaving
- Single interleaving
- Double interleaving

Performance Measures of Disks (1)

- **Access time** – the time it takes from when a read or write request is issued to when data transfer begins. Consists of:
  - **Seek time** – time it takes to reposition the arm over the correct track.
    - Average seek time is 1/2 the worst case seek time.
      - Would be 1/3 if all tracks had the same number of sectors, and we ignore the time to start and stop arm movement
    - 4 to 10 milliseconds on typical disks
  - **Rotational latency** – time it takes for the sector to be accessed to appear under the head.
    - Average latency is 1/2 of the worst case latency.
    - 4 to 11 milliseconds on typical disks (5400 to 15000 r.p.m.)

Performance Measures of Disks (2)

- **Data-transfer rate** – the rate at which data can be retrieved from or stored to the disk.
  - 4 to 8 MB per second is typical
- **Mean time to failure (MTTF)** – the average time the disk is expected to run continuously without any failure.
  - Typically 3 to 5 years

Disk Arm Scheduling Algorithms (1)

- Time required to read or write a disk block determined by 3 factors
  - Seek time
    - Move arm to the correct cylinder
  - Rotational delay
    - Wait until correct sector comes under head.
  - Actual data transfer time
- For most systems, the seek time dominates the other two times.

Disk Arm Scheduling Algorithms (2)

- Assume the disk is heavily loaded.
  - There are a lot requests for disk blocks that are arriving to the hard disk driver.
  - The drivers queues the requests.
    - The driver knows for each block request, where in hard disk the block should be stored (the cylinder number).
  - Since moving between cylinders (seek time) is costly, we should try to minimize these seek times.

Disk Arm Scheduling Algorithms (3)

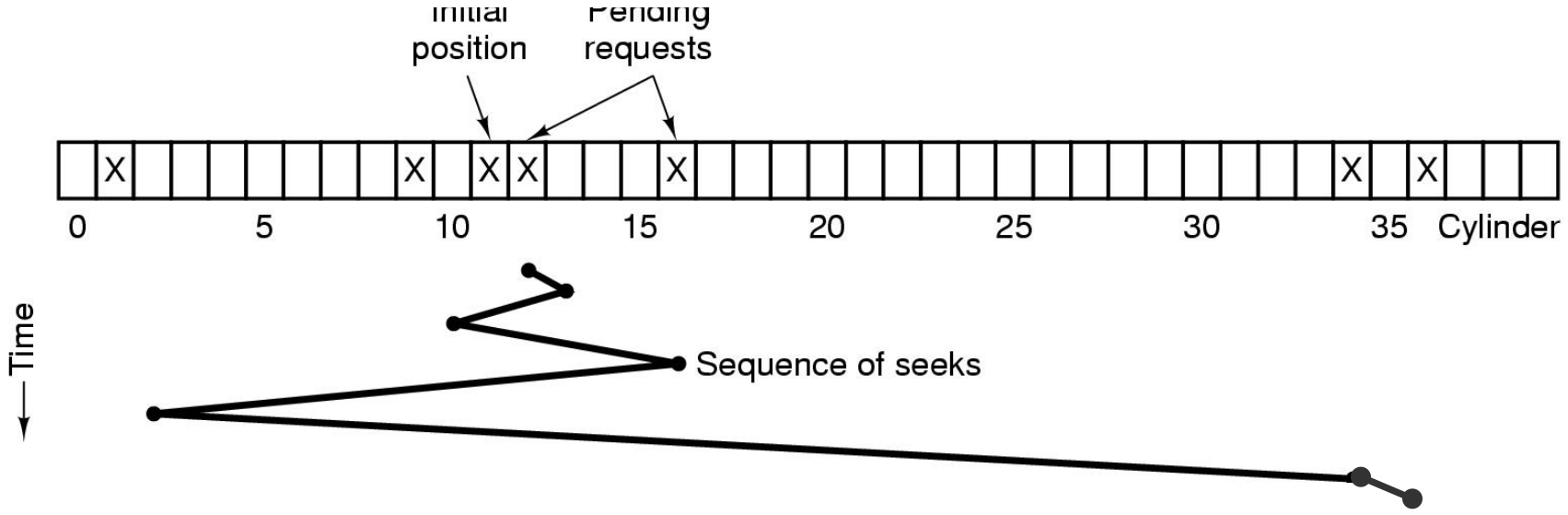
- Algorithms order pending accesses to tracks so that disk arm movement is minimized
  - First Come First Served (FCFS)
  - Shortest Seek First (SSF)
  - Elevator Algorithm

FCFS

- Serve the request in the order they arrive (don't look to the cylinder numbers)
- Example:
  - Assume the current position is cylinder 11.
  - The following requests arrive in the given time order: 1, 36, 16, 34, 9, 12
  - The request will be served in the same order.
  - The head will go to:
    - Cylinder 1 first (10 cylinders motion)
    - Cylinder 36 next (35 cylinders motion)
    - Cylinder 16 next (20 cylinders motion)
    - Cylinder 34 next (18 cylinders motion)
    - Cylinder 9 next (25 cylinders motion)
    - Cylinder 12 next (3 cylinders motion)
    - Total of 111 cylinders are skipped. Cost = 111

Shortest Seek First (SSF)

- Satisfy the request that need shortest seek from the current position.
  - Cylinder request sequence: 11、1、36、16、34、9、12
  - Sequence of seeks: 11、12、9、16、1、34、36
  - Moving the head to these cylinders require: 1, 3, 7, 15, 33, and 2 arm motions, total cost=61

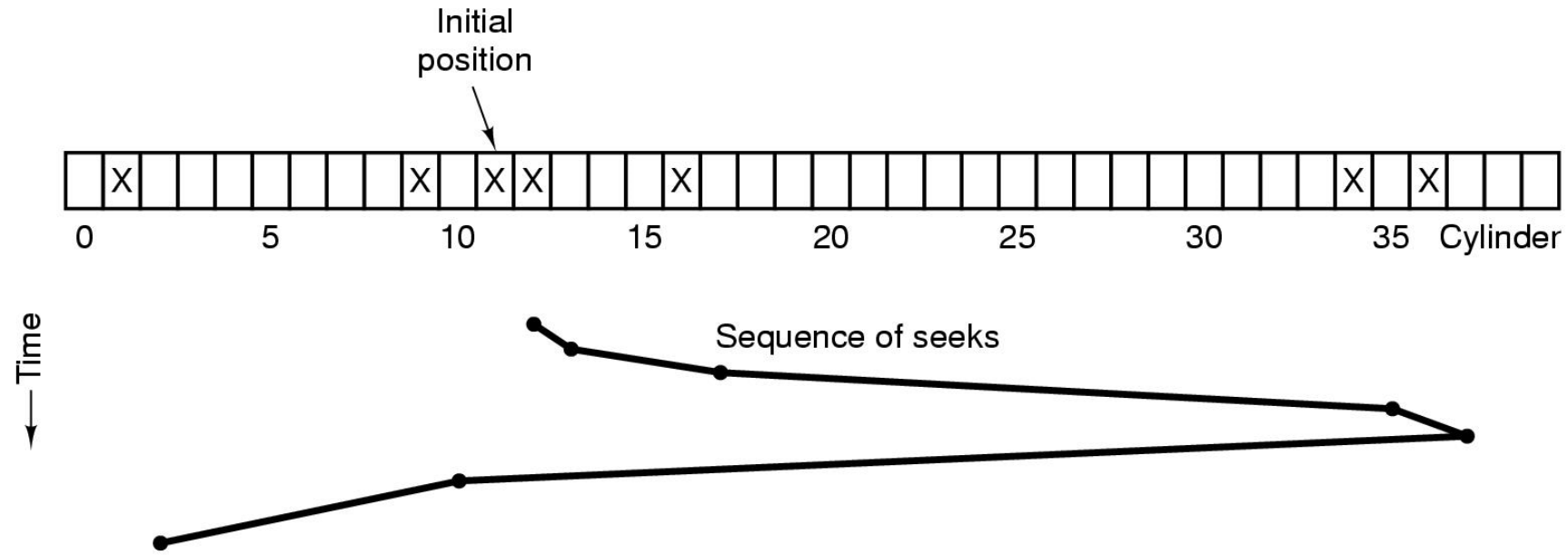


Elevator algorithm (1)

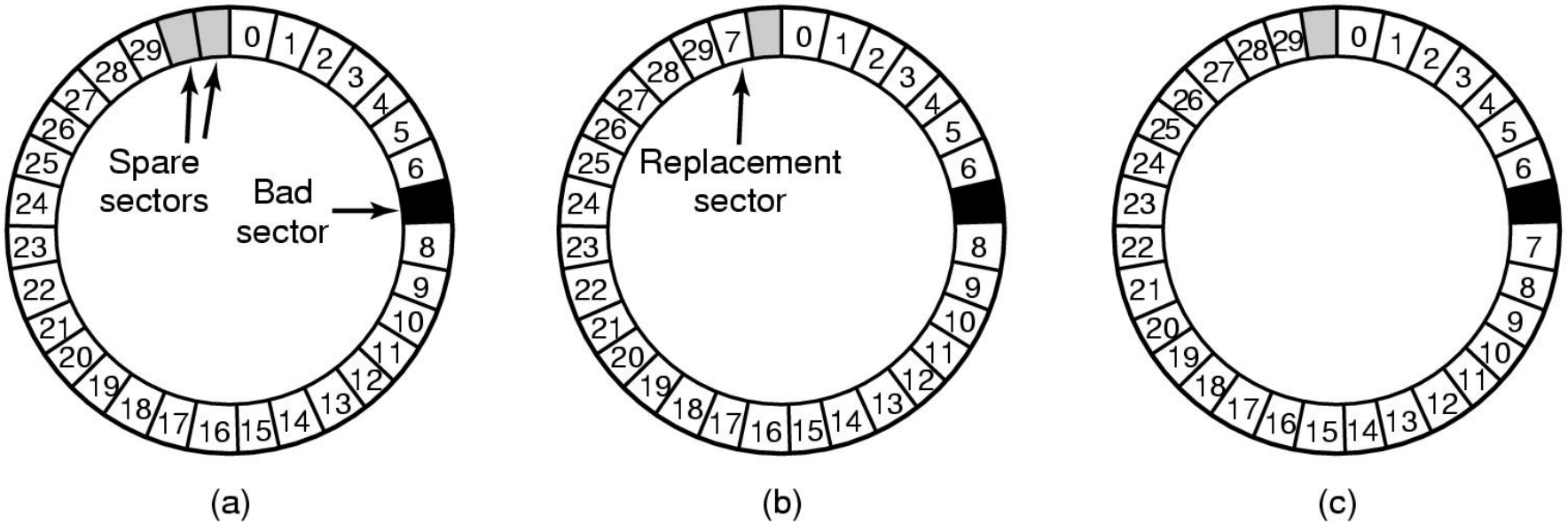
- Elevator algorithm: Move disk arm in one direction (from outer to inner tracks or vice versa), processing next request in that direction, till no more requests in that direction, then reverse direction and repeat
- In the previous example, assume initial position was 11 and initial direction was up (going to higher numbered cylinders).

Elevator algorithm (2)

- Cylinder request sequence: 11、1、36、16、34、9、12
- Sequence of seeks: 11、12、16、34、36、9、1
- Moving the head to these cylinders require: 1, 4, 18, 2, 27, and 8 arm motions, total cost=60



Error Handling



- A disk track with a bad sector
- Substituting a spare for the bad sector
- Shifting all the sectors to bypass the bad one

Summary

- Principles of I/O Hardware
  - I/O Devices
  - Separate I/O and memory space, Memory-mapped I/O, Hybrid
- Principles of I/O Software
  - Goals of I/O software
  - Programmed I/O
  - Interrupt driven I/O
  - I/O using DMA
- I/O software layers
- Disks
  - Disk formatting
  - Disk arm scheduling algorithm